

Lecture 6

Cleaning and Normalization

Unicode, case, stop words, stemming, lemmatization — and when to do none of it

café ≠ café until you normalize

Two strings that look identical are not equal in memory. Cleaning decides what counts as the same word.

What Lecture 6 covers

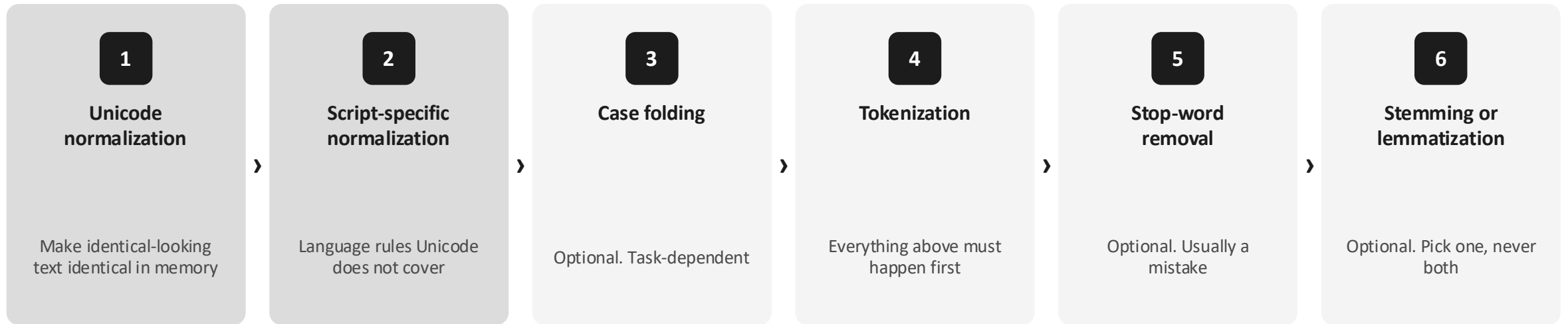
Cleaning and normalization, in order

We take the tokens from Lecture 5 and ask what should count as the same token. That means Unicode normalization, script-specific fixes Unicode does not cover, case folding, stop-word removal, and the difference between stemming and lemmatization. For each one, the real lesson is when NOT to do it, because most of these were designed for sparse count-based models and hurt a pretrained model.

One numerical you should be able to reproduce in the exam

- 1 Vocabulary reduction from case folding
- 2 A Porter stemmer trace, step by step
- 3 Stemming versus lemmatization on the same words

Cleaning and normalization, in the order it must run



Two rules about this pipeline

Order is not negotiable for steps 1 to 4

Normalize before you tokenize. If you tokenize first, the same word in two encodings becomes two different types, and no later step can merge them.

Steps 3, 5 and 6 are optional and often harmful

They were designed for sparse count-based models. With a pretrained model, the default answer is to do none of them, because the model was pretrained on raw text.

Unicode: when identical-looking text is not identical

THE PROBLEM, IN CODE

```
s1 = "caf\u00e9"      # e-acute as one codepoint   U+00E9
s2 = "cafe\u0301"     # e plus combining acute    U+0065 U+0301

print(s1, s2)
café café           ← they render identically
len(s1), len(s2)
(4, 5)              ← but they are not the same string
s1 == s2
False               ← two vocabulary entries for one word
```

NFC Composed

Combine base + mark into a single codepoint where one exists.

café → café (5 → 4)

NFD Decomposed

Split precomposed characters into base plus combining marks.

café → café (4 → 5)

NFKC Compatibility composed

Also folds compatibility variants: ligatures, superscripts, enclosed forms.

file → file ① → 1

NFKD Compatibility decomposed

Compatibility folding plus decomposition. The most aggressive of the four.

Loses formatting distinctions

Default advice: apply NFC to everything on the way in. Use NFKC only when you are sure the compatibility folding is harmless for your task.

What Unicode normalization will not fix

NFC merges encodings of the same character. It does not merge different characters that look alike.

Yeh variants

Arabic Yeh U+064A and Farsi/Urdu Yeh U+06CC are distinct codepoints that render similarly in many fonts. Text from Arabic-configured keyboards mixes them freely.

Needs a custom mapping table

Kaf variants

Arabic Kaf U+0643 and Keheh U+06A9 have the same problem. Urdu convention is Keheh; a great deal of real text uses Kaf.

Needs a custom mapping table

Optional diacritics

Zabar, zer, pesh and other harakat may be present or absent in otherwise identical words, producing two types where there should be one.

Strip or keep, but do it consistently

Space behaviour

The script joins letters, so writers insert zero-width non-joiners and omit or add spaces inconsistently. Whitespace is not a reliable word delimiter.

Handle before tokenization

Libraries such as urduhack ship these mappings. If you write your own, document it and version it, because your vocabulary depends on it.

Case folding, and what it costs you

WORKED EXAMPLE · VOCABULARY REDUCTION

Observed forms in a small corpus:

The, the, THE, Apple, apple, Cat, cat

V before = 7 types
V after = 3 types {the, apple, cat}
reduction = $(7 - 3) / 7 = 4/7 = 57.1\%$

On real corpora, lowercasing typically removes a smaller but still substantial share of types, concentrated in sentence-initial words.

WHAT YOU DESTROY

Apple	vs	apple	Company or fruit		
US	vs	us	Country or pronoun		
Turkey	vs	turkey	Country or bird		
MAY	vs	May	vs	may	Shouting, month, or modal verb

When to fold, and when not to

Fold

Information retrieval, topic classification, and any bag-of-words model where sparsity is the binding problem.

Do not fold

Named entity recognition, where capitalisation is one of the strongest features available in English.

Irrelevant

Scripts without case, including Urdu, Arabic, Chinese and Devanagari. The decision simply does not arise.

Stop-word removal, and the trap inside it

THE IDEA, AND THE ARITHMETIC

Zipf's law says a few dozen function words account for a large share of all tokens. Removing them shrinks the feature space at little apparent cost.

Sentence: "The movie was not good at all"

tokens before = 7
removed = the, was, not, at, all (5)
tokens after = 2 → 71% reduction

WHAT SURVIVED

movie

good

A sentiment classifier now sees a positive review. The original was negative. The standard NLTK English stop-word list contains 'not', along with 'no', 'nor' and other negations, so this failure is the default behaviour, not an edge case.

Practical guidance

Retrieval and topic models

Removal is usually fine and sometimes helps.
Function words carry little topical signal.

Sentiment, NLI, QA

Do not remove. Negation, modality and quantifiers are exactly the function words on the list.

Any pretrained model

Never remove. The model was pretrained on text containing them and its syntax depends on them.

The Porter stemmer: rules applied in five steps

THE MEASURE M: THE ENGINE BEHIND EVERY CONDITION

Write the word as [C] (VC)^m [V]. m is the number of VC sequences. Conditions like (m>0) or (m>1) stop the stemmer chewing short words down to nothing.

tr, ee, tree, by → m = 0 trouble, oats, trees, ivy → m = 1 troubles, private, oaten → m = 2

1a	Plurals	SSES→SS, IES→I, SS→SS, S→∅	caresses→caress, ponies→poni, cats→cat
1b	Past and progressive	(m>0) EED→EE; (*v*) ED→∅; (*v*) ING→∅	agreed→agree, plastered→plaster, motoring→motor
1c	Terminal y	(*v*) Y → I	happy→happi, sky→sky (no vowel before y)
2, 3	Long suffixes	(m>0) ATIONAL→ATE, TIONAL→TION, ICAL→IC, FUL→∅, NESS→∅	relational→relate, electrical→electric
4	Strip remaining suffixes	(m>1) AL, ANCE, ENCE, ER, IC, ABLE, ANT, MENT, ENT, ION, ATE, IVE, IZE → ∅	adjustment→adjust, defensible→defens
5	Tidy up	(m>1) E→∅; (m>1, *d, *L) double consonant → single	probate→probat, controll→control

Tracing two words through the algorithm

RELATIONAL

1a no rule applies relational

1b no eed/ed/ing relational

1c no terminal y relational

2 ATIONAL→ATE, m(rel)=1>0 relate

3 no rule applies relate

4 ATE needs m>1; m(rel)=1 relate

5a E deleted, m(relat)=2>1 relat

5b no double consonant relat

Output: relat not a word, and that is acceptable

STUDYING

1a no rule applies studying

1b (*v*) ING deleted study

1b+ not a double consonant study

1c (*v*) Y→I studi

2 no rule applies studi

3 no rule applies studi

4 no rule applies studi

5 no rule applies studi

Output: studi same stem as studies, study, studied

Stemming against lemmatization

WORD	PORTER STEM	WORDNET LEMMA	COMMENT
studies	studi	study	Both collapse it correctly
studying	studi	study (as a verb)	Lemma needs the part of speech
better	better	good (as an adjective)	Stemmer cannot handle suppletion
was	wa	be (as a verb)	Stemmer produces a non-word
universities	univers	university	Stemmer over-reaches and merges 'universe'
mice	mice	mouse	Irregular plural; rules alone cannot do this

Vocabulary reduction on {studies, studying, study, studied}: stemming 4 → 1 types (75%), lemmatization 4 → 1 types (75%). Same reduction here; the difference is that the lemma is a real word and the stem is not.

Hands on: normalization end to end

UNICODE

```
import unicodedata as ud
s1, s2 = "caf\u00e9", "café\u0301"

len(s1), len(s2), s1 == s2
(4, 5, False)

n = lambda x: ud.normalize("NFC", x)
n(s1) == n(s2)
True
```

STEMMING AND LEMMATIZATION

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
ps, wnl = PorterStemmer(), WordNetLemmatizer()
ws = ["studies", "studying", "study", "studied"]

[ps.stem(w) for w in ws]
['studi', 'studi', 'studi', 'studi']

[wnl.lemmatize(w, pos="v") for w in ws]
['study', 'study', 'study', 'study']
```

PUTTING THE PIPELINE TOGETHER, AND MEASURING WHAT IT DID

```
def preprocess(text, fold=True, stem=True):
    text = ud.normalize("NFC", text)          # 1. always
    if fold: text = text.lower()              # 2. task-dependent
    toks = word_tokenize(text)                # 3. after normalizing
    if stem: toks = [ps.stem(t) for t in toks] # 4. optional
    return toks

len(set(word_tokenize(corpus))), len(set(preprocess(corpus)))
1 - len(set(preprocess(corpus))) / len(set(word_tokenize(corpus)))
```

Eight ways this goes wrong in practice

1 Tokenizing before normalizing

Two encodings become two types and nothing later can merge them.

3 Lowercasing text for a cased model

Throws away a feature the model was pretrained to use.

5 Stemming and lemmatizing

Pick one. Doing both produces incoherent forms.

7 Different preprocessing at train and test

The most common silent failure in student projects. Wrap it in one function and call that function everywhere.

2 Mismatched model and tokenizer

No error is raised. Ids map to the wrong embeddings and output is confident nonsense.

4 Removing stop words before sentiment

Standard lists contain 'not'. The label inverts.

6 Comparing TTR across different lengths

TTR falls with corpus size. The comparison is meaningless without a fixed window.

8 Not recording what you did

Vocabulary size, sequence length and every metric depend on these choices. Log them with the results.

Exercises

1 Types and tokens

For the corpus “the cat sat on the mat the cat ran”, compute N, V and the TTR.

$N = 9, V = 6, TTR = 0.667$

3 BPE training

Run three merges on: $4 \times (b \ a \ _)$, $3 \times (b \ a \ n \ _)$, $2 \times (b \ a \ t \ _)$. Give the merge list.

$(b, a); (ba, n); (ban, _)$ at counts 9, 7, 4

5 Fertility

A 12-word Urdu sentence tokenizes to 31 pieces. Give the fertility and say what it implies for cost.

2.58; roughly $2.6 \times$ the tokens per word

2 Heaps' law

A corpus of 500,000 tokens has 35,000 types. With $\beta = 0.5$, estimate K and predict V at 2 million tokens.

$K \approx 49.5; V \approx 70,000$

4 BPE encoding

Using the eight-merge table from the lecture, segment “lowest”.

$low + e + s + t + _$ (5 tokens)

6 Porter

Trace “conditional” through the algorithm.

TIONAL→TION gives condition; step 4 ION with $m > 1$ gives condit

What to take away, and what comes next

1 **Tokenization fixes your vocabulary, your sequence length and your costs** Every number you report downstream is conditional on it, so record it.

2 **Subword units solve the open-vocabulary problem** Bounded vocabulary, no UNK, and morphology emerges for free where morphology is frequent.

3 **BPE is merge the most frequent pair, k times** The ordered merge list is the model. Vocabulary size is characters plus k.

4 **Normalize before you tokenize, always** And remember that Unicode normalization does not merge different characters that merely look alike.

5 **Case folding, stop words and stemming are optional and usually wrong now** They were designed for sparse count models. With a pretrained model, do none of them.

Next: Lectures 7 and 8, Word Representations. Once text is in tokens, the question becomes how to turn a token into a vector.
Reading: Jurafsky & Martin, Chapter 2 (regular expressions, text normalization, edit distance), and Sennrich et al. (2016) on subword units.